

Pixbe CRM

Step-by-step AWS Implementation Plan

ECS Fargate + ALB + Cognito + Aurora PostgreSQL (per-workspace databases). Cloudflare proxy in front of the ALB. Public subnets only (no private subnets / no NAT). Region: ap-south-1 (Mumbai). No EC2 compute, no Redis, no Zapier.

1. Locked stack

Piece	Choice
Compute	ECS Fargate (no EC2 app servers)
Load balancer	Application Load Balancer (ALB), no nginx
Edge / DNS	Cloudflare proxy (orange-cloud) in front of ALB
Network	Public subnets only - no private subnets, no NAT gateway
Security groups	Anywhere (0.0.0.0/0) on required ports
Auth (humans)	Amazon Cognito + JWT-only tenant binding
Auth (website ingest)	API keys per workspace (custom HTTP only)
Data	Aurora PostgreSQL: pixbe_control (memberships/billing/API keys) + one DB per workspace (CRM agents/leads)
Region	ap-south-1 (Mumbai)
Deferred	Redis, ECS on EC2, Zapier/Pabbly, TeleCRM Chrome extension

2. Architecture (summary)

Browser and website forms -> Cloudflare proxy -> ALB -> ECS Fargate (public subnets). Users authenticate with Cognito. App uses RDS Proxy to Aurora also in public subnets. Aurora hosts pixbe_control (directory/billing/API keys/memberships) and one database per workspace (e.g. pixbe_company_kite_aviation for CRM data). Creating a workspace provisions a new workspace DB. It does not become the source of truth for login. db_company_acme in diagrams is only a fictional second-customer example. Security groups allow 0.0.0.0/0 (anywhere) on the ports the stack needs.

3. Where workspace user data is stored

Admin, Manager, and Telecaller in the same workspace share one workspace database. They are different roles, not different databases. User data is split: login identity lives in Cognito; membership lives in the shared control plane; CRM agent records live in the individual workspace DB.

Data	Where	Why
Email, password, Cognito user id	Amazon Cognito	Human auth. Passwords are not stored in either Postgres database.
Workspace list, billing, API keys, routing	pixbe_control (shared)	
Membership: user X belongs to workspace Y as Admin / Manager / Telecaller	pixbe_control.memberships	Lets Fargate resolve JWT -> tenant without scanning every workspace DB.
CRM agent profile: name, phone, manager hierarchy, assignment, call stats	That workspace's database	Same isolated DB as leads, tasks, and calls for that company.

How a login works

1. User authenticates with Cognito.

2. JWT carries custom:tenant_id and custom:role, or the app looks up membership by Cognito sub in pixbe_control.
3. App opens only that workspace DB and loads CRM data (leads, tasks, and the agent row used for assignment).

Practical rules:

- Can this user enter this workspace, and as which role? -> pixbe_control + Cognito.
- What this user owns, calls, converts, and reports inside the CRM? -> that workspace's database.
- Do not store passwords in the workspace DB.
- Do not put leads, tasks, or call logs in pixbe_control.
- Signup flow: Cognito sign-up -> provision tenant/workspace DB -> write membership in pixbe_control.

4. What to create first - and why

Order: Database (local then Aurora) -> Cognito -> API keys -> Server (Fargate).

Order	Create	Why
1st	Database layer	JSON today; need SQL + per-workspace DBs before Cognito signup or multi-task Fargate.
2nd	Cognito	Needs control-plane memberships / tenant_id in JWT. Auth before DB has nowhere to attach workspaces.
3rd	API keys + ingest	Stored in control plane; writes into workspace DB. Testable without Fargate.
4th	ECS Fargate + ALB	Needs DB endpoints, Cognito IDs, secrets. Deploying server first with JSON/sessions causes rework.

Benefits of this order:

- Fail fast on JSON->SQL before paying for always-on AWS.
- Auth/tenancy correct before running 2+ containers behind ALB.
- Clear exit test per step.
- Cutover is mostly DNS + data import.

5. Step-by-step process

Step 1 - Local database + app rewrite (first)

Goal: Postgres with control plane + one DB per workspace on Docker Compose.

1. CREATE DATABASE pixbe_control and pixbe_<tenant> scripts.
2. Migrations: control plane (tenants, memberships, api_keys, routing) + workspace schema (agents, leads, tasks, calls). Memberships stay in pixbe_control; agent CRM profiles stay in the workspace DB.
3. Implement controlPlane.ts, tenantPool.ts, provisionTenant.ts.
4. Rewrite multiTenantDb to use tenant pool (not JSON file).
5. JSON -> Postgres import script per tenant.
6. Prove: compose up, create tenant, leads CRUD, /api/health.

Benefit: Zero AWS cost; same path as Aurora. Exit: no production path on multi_tenant_store.json.

Step 2 - AWS network foundation (ap-south-1)

Goal: Public-subnet VPC for Aurora and Fargate in Mumbai. No private subnets and no NAT gateway. Cloudflare sits in front of the ALB.

1. VPC, 2 AZs, public subnets only (no private subnets, no NAT).
2. Security groups with Anywhere (0.0.0.0/0): ALB 80/443 from 0.0.0.0/0; app 8080 from 0.0.0.0/0 (or from

ALB SG if preferred later); Aurora/Proxy 5432 from 0.0.0.0/0; HTTPS egress allowed.

3. ACM certificate for the origin domain the ALB presents to Cloudflare.

4. IAM roles for ECS task execution and task role.

5. Cloudflare: add DNS A/CNAME to ALB, enable proxy (orange cloud), SSL/TLS Full (strict) once ACM is valid.

Benefit: Simpler network for this stage; edge TLS and DDoS via Cloudflare. Exit: VPC + SGs + Cloudflare DNS ready, no app deployed yet.

Step 3 - Aurora (cloud database)

Goal: Same DB model as Step 1 on AWS.

1. Aurora PostgreSQL (Serverless v2 OK) in public subnets (no private subnets).

2. RDS Proxy + Secrets Manager.

3. SG for Postgres: allow 5432 from Anywhere 0.0.0.0/0 (per locked stack).

4. Create pixbe_control; run migrations.

5. Provision test workspace DB; import sample tenant.

6. Point app at Aurora with SSL.

Benefit: Durable data before auth/server. Why not Cognito/Fargate yet: signup creates DBs; Fargate needs secrets.

Step 4 - Cognito + JWT-only tenant binding

Goal: Humans use Cognito; tenant only from verified JWT.

1. User Pool + App Client (SPA + PKCE).

2. custom:tenant_id, custom:role and/or memberships by Cognito sub in pixbe_control.memberships.

3. Register: Cognito sign-up -> provisionTenant (new workspace DB) -> membership row in pixbe_control (role Admin / Manager / Telecaller) -> agent row in that workspace DB.

4. Replace session Map with JWT verify (JWKS).

5. Stop trusting bare x-tenant-id; allow only with membership proof.

6. Update client auth; real Navbar workspace switch.

7. Disable hardcoded demo admin in production.

Benefit: Stateless auth for multi-task Fargate; no header spoofing.

Step 5 - API keys (website / custom HTTP only)

Goal: Website forms or custom backends push leads. Zapier/Pabbly out of scope.

1. api_keys table - store hash only; show raw key once.

2. Admin UI: create / revoke / rotate.

3. POST /api/v1/ingest/leads with Bearer pk_... or X-API-Key.

4. Resolve key -> tenant -> workspace DB; audit without full PII.

5. Short docs for calling ingest from your site/server.

Exit: API key creates a lead only in that workspace DB.

Step 6 - Container image (ECR)

1. ECR repo pixbe-crm in ap-south-1.

2. Build Dockerfile; healthcheck /api/health.

3. Push version tag and latest.

Step 7 - ALB + ECS Fargate (server last)

Goal: Public HTTPS on Fargate behind Cloudflare - no EC2, no private subnets.

1. ALB in public subnets; HTTPS + ACM; SG allows 80/443 from Anywhere 0.0.0.0/0; target group port 8080; health /api/health.
2. ECS cluster + Fargate task (ECR image; secrets for DB, Cognito, Gemini, Meta, Razorpay).
3. Service desired count 2+, public subnets only, assign public IPs as needed, attached to ALB. Task SG: 8080 from Anywhere 0.0.0.0/0 (locked stack).
4. Optional SQS worker for Meta webhooks / workflows.
5. CloudWatch. Prefer Cloudflare proxy for edge protection instead of AWS WAF for now.

Why last: Needs Steps 3-6. Exit: <https://your-domain/api/health> OK via Cloudflare; Cognito + API-key ingest via Cloudflare -> ALB.

Step 8 - Data cutover + go-live

1. Freeze or final-export JSON.
2. Import all tenants; verify counts.
3. Point Cloudflare DNS at the ALB (proxied / orange cloud). Confirm SSL mode Full (strict).
4. Smoke: Cognito, leads, Meta webhook, API-key ingest, health through Cloudflare.
5. Keep JSON backup for short rollback window.
6. Retire EB zip / JSON as primary path.

Step 9 - Later (not blocking go-live)

TeleCRM parity later: one IVR/telephony vendor, WhatsApp Cloud API depth, IndiaMART/Justdial-style connectors.

6. Printable checklist

- [] 1. Local Postgres + control plane + per-workspace DBs + memberships vs agents split + multiTenantDb SQL + import
- [] 2. VPC public subnets only + SGs 0.0.0.0/0 + ACM + Cloudflare proxy DNS (ap-south-1)
- [] 3. Aurora (public subnets) + RDS Proxy + Secrets (+ test tenant)
- [] 4. Cognito + JWT-only tenant binding (+ workspace switch)
- [] 5. API keys + /api/v1/ingest (website/custom HTTP)
- [] 6. ECR image push
- [] 7. ALB + ECS Fargate in public subnets (+ SQS workers)
- [] 8. Import + Cloudflare DNS cutover + smoke tests
- [] 9. (Later) TeleCRM feature parity

7. Key files

- server/services/multiTenantDb.ts
- server/middleware/auth.ts, tenantContext.ts
- src/lib/auth.ts, Navbar.tsx
- New: server/db/*, server/modules/apiKeys/*, server/modules/ingest/*
- docker-compose.yml
- New infra/: VPC (public only), Aurora, Proxy, Cognito, ECR, ALB, ECS Fargate, SQS, Secrets, Cloudflare DNS

8. Out of scope now

- EC2 for compute or as load balancer
- Private subnets and NAT gateway
- Redis
- Zapier / Pabbly
- One Aurora cluster per customer
- Full TeleCRM Chrome extension in Step 8
- Region ap-south-2 - use ap-south-1 Mumbai only